

DOCUMENTAÇÃO TÉCNICA E RELATÓRIO ACADÊMICO

Projeto: Mini Gestão de Produtos

Sistema Web para Gestão de Fornecedores, Produtos e Cesta de Compras

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Desenvolvimento de Sistemas Web (PHP / MySQL)

Tecnologias: PHP 8 (POO + PDO) • MySQL/MariaDB • Bootstrap 5 • JavaScript (Fetch API)

Goioerê – Paraná

2026

SUMÁRIO

1. RESUMO E INTRODUÇÃO

1.1 Resumo

Este relatório apresenta a documentação técnica e acadêmica completa do sistema “Mini Gestão de Produtos”, uma aplicação web desenvolvida em PHP e MySQL cujo propósito é oferecer, de forma simples e didática, um fluxo completo de gestão de fornecedores, produtos e seleção de itens em uma cesta de compras. O sistema foi concebido como um estudo de caso de arquitetura orientada a objetos no back-end, aliado a uma camada de front-end responsiva e interativa, comunicando-se de forma assíncrona com o servidor.

O documento está organizado em seis capítulos, cobrindo desde a fundamentação teórica e as tecnologias empregadas até a modelagem de dados, o fluxo de utilização da aplicação e as considerações finais sobre o cumprimento dos requisitos propostos.

1.2 Objetivo do Sistema

O “Mini Gestão de Produtos” tem como objetivo central oferecer uma solução web simplificada, porém funcionalmente completa, para três domínios de negócio interligados:

- Gestão de Fornecedores: cadastro, edição, listagem e exclusão de empresas fornecedoras de produtos;
- Gestão de Produtos: cadastro de produtos vinculados a um fornecedor específico, com preço, descrição e controle de CRUD completo;
- Cesta de Compras: seleção de produtos via checkbox, com cálculo automático do valor total em Reais (R\$), simulando um carrinho de compras simplificado.

O sistema foi projetado para ser executado em ambiente local (XAMPP, WAMP, Laragon ou similar), com criação automática do banco de dados e das tabelas no primeiro acesso, eliminando a necessidade de scripts SQL manuais para o usuário final.

1.3 Visão Geral da Arquitetura

A arquitetura do sistema segue o paradigma de Programação Orientada a Objetos (POO) em PHP, organizando as responsabilidades em classes coesas e de baixo acoplamento. Os principais pilares arquiteturais adotados são:

- Orientação a Objetos: cada entidade de negócio (Usuário, Fornecedor, Produto, Cesta) é representada por uma classe PHP própria, encapsulando dados e regras de negócio;
- Padrão Singleton: a classe Database.php garante uma única instância de conexão PDO ativa durante o ciclo de vida da requisição, evitando múltiplas conexões redundantes com o banco;
- Comunicação Assíncrona (AJAX/Fetch API): as operações de cadastro, edição, exclusão e seleção de itens são realizadas via requisições assíncronas com a Fetch API nativa do JavaScript, sem recarregamento de página, resultando em uma experiência de uso mais fluida;
- Separação de Responsabilidades (Camadas): o projeto separa claramente a camada de apresentação (HTML/CSS/JS), a camada de acesso a dados (classes PHP + PDO) e os endpoints de comunicação (scripts na pasta /ajax), aproximando-se de um modelo em camadas inspirado no padrão MVC.

Essa combinação resulta em um sistema de baixa complexidade de infraestrutura (não exige frameworks pesados), mas que demonstra, de forma prática, conceitos fundamentais de engenharia de software como abstração, encapsulamento, reuso de código e segurança no acesso a dados.

2. TECNOLOGIAS UTILIZADAS

A tabela a seguir resume a stack tecnológica adotada no desenvolvimento do sistema:

Camada	Tecnologia	Finalidade no Projeto
Back-end	PHP 8+ (POO) com PDO	Lógica de negócio, orientação a objetos e acesso seguro ao banco de dados
Banco de Dados	MySQL / MariaDB	Persistência de dados com criação automática de schema
Front-end (Estrutura)	HTML5	Marcação semântica das páginas e formulários
Front-end (Estilo)	CSS3 + Bootstrap 5	Design responsivo, componentes visuais (cards, tabelas, modais, alerts)
Front-end (Interatividade)	JavaScript nativo (Fetch API)	Requisições assíncronas (AJAX) sem recarregamento de página
Segurança	SHA-256 com salt fixo, Prepared Statements, auth_check.php	Proteção de senhas, prevenção de SQL Injection e controle de sessão

Tabela 2.1 — Stack tecnológica do sistema Mini Gestão de Produtos

2.1 Linguagem de Programação: PHP com PDO

O back-end foi inteiramente desenvolvido em PHP, utilizando a extensão PDO (PHP Data Objects) para comunicação com o banco de dados. A escolha do PDO, em detrimento das antigas funções mysqli procedurais, se justifica por permitir:

- Uso de Prepared Statements (declarações preparadas), que separam o código SQL dos dados de entrada;
- Portabilidade entre diferentes SGBDs (MySQL, MariaDB, PostgreSQL, SQLite) com mínima alteração de código;
- Tratamento de exceções via try/catch (PDOException), tornando o tratamento de erros mais robusto e previsível;
- Suporte nativo a transações (beginTransaction, commit, rollBack), úteis em operações que envolvem múltiplas tabelas.

2.2 Banco de Dados: MySQL / MariaDB

O sistema utiliza MySQL (ou MariaDB, seu fork compatível) como Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR). Um diferencial de usabilidade do projeto é a criação automática do banco de dados e das tabelas no primeiro acesso: a classe Database.php verifica a existência do schema mini_gestao_produtos e, caso não exista, executa os comandos CREATE DATABASE e CREATE TABLE necessários, tornando o processo de instalação transparente para o usuário/avaliador.

2.3 Front-end: HTML5, CSS3, Bootstrap 5 e JavaScript

A interface do usuário foi construída com HTML5 semântico, estilizada com CSS3 e o framework Bootstrap 5, garantindo responsividade em diferentes tamanhos de tela (desktop, tablet e mobile) através do sistema de grid e dos componentes prontos do framework (cards, modais, tabelas, alerts e formulários). A interatividade — cadastro, edição, exclusão e seleção de produtos — é implementada em JavaScript nativo (sem jQuery ou frameworks adicionais), utilizando a Fetch API para realizar requisições HTTP assíncronas (AJAX) aos scripts PHP localizados na pasta /ajax, atualizando o DOM dinamicamente a partir das respostas em formato JSON.

2.4 Segurança da Aplicação

Foram adotadas três frentes complementares de segurança:

- Hash de senhas (SHA-256 + salt fixo): as senhas dos usuários nunca são armazenadas em texto puro. Antes da persistência, a senha é concatenada a um salt fixo definido em configuração e submetida à função hash('sha256', ...), dificultando ataques de força bruta e rainbow tables simples;
- Prepared Statements (PDO): todas as consultas SQL que recebem dados do usuário utilizam parâmetros nomeados (:param) ou posicionais (?), eliminando a possibilidade de SQL Injection por concatenação direta de strings;
- Verificação de sessão (auth_check.php): um script de guarda é incluído no topo de todas as páginas e endpoints protegidos, validando se existe uma sessão PHP ativa e válida (via \$_SESSION) antes de permitir o acesso; caso contrário, o usuário é redirecionado para a tela de login.

3. ESTRUTURA DO PROJETO E MÓDULOS

3.1 Árvore de Diretórios

A organização física dos arquivos do projeto segue uma separação lógica por responsabilidade, facilitando a manutenção e a localização de cada componente:

```
mini-gestao-produtos/
├── ajax/                # Endpoints assíncronos (recebem requisições Fetch/AJAX)
│   ├── fornecedor_salvar.php
│   ├── fornecedor_excluir.php
│   ├── produto_salvar.php
│   ├── produto_excluir.php
│   └── cesta_atualizar.php
├── assets/             # Recursos estáticos do front-end
│   ├── css/
│   │   └── style.css
│   └── js/
│       ├── fornecedores.js
│       ├── produtos.js
│       └── cesta.js
├── auth/               # Autenticação e controle de sessão
│   ├── login.php
│   ├── registrar.php
│   ├── logout.php
│   └── auth_check.php
├── classes/            # Camada de domínio (POO)
│   ├── Database.php
│   ├── Usuario.php
│   ├── Fornecedor.php
│   ├── Produto.php
│   └── Cesta.php
├── config/             # Configurações gerais (credenciais, constantes)
│   └── config.php
├── includes/           # Componentes reutilizáveis de layout
│   ├── header.php
│   └── footer.php
├── index.php           # Tela inicial / dashboard
├── fornecedores.php    # Tela de gestão de fornecedores
├── produtos.php        # Tela de gestão de produtos
└── cesta.php           # Tela de resumo da cesta de compras
```

Figura 3.1 — Estrutura de diretórios do projeto

3.2 Classes Principais (Camada de Domínio)

3.2.1 Database.php

Responsável pela conexão com o banco de dados, implementada segundo o padrão de projeto Singleton. Essa classe garante que, independentemente de quantas vezes for solicitada uma conexão ao longo da requisição, apenas uma única instância de PDO seja criada e reaproveitada, reduzindo overhead de recursos.

- Método estático getInstance(): retorna a instância única da conexão, criando-a apenas na primeira chamada;
- Auto-inicialização do banco: na primeira execução, a classe verifica e cria automaticamente o banco `mini_gestao_produtos` e as tabelas necessárias, caso ainda não existam, via comandos DDL (CREATE DATABASE IF NOT EXISTS / CREATE TABLE IF NOT EXISTS);
- Configuração de atributos PDO: define PDO::ATTR_ERRMODE como PDO::ERRMODE_EXCEPTION, garantindo que erros de banco sejam lançados como exceções tratáveis.

3.2.2 Usuario.php

Classe responsável pelo domínio de autenticação. Concentra as regras de negócio relacionadas ao ciclo de vida do usuário no sistema:

- registrar(nome, email, senha): valida e insere um novo usuário na tabela `usuarios`, aplicando o hash SHA-256 com salt fixo sobre a senha antes da persistência;
- autenticar(email, senha): recalcula o hash da senha informada e compara com o valor armazenado, retornando os dados do usuário em caso de sucesso e iniciando a sessão PHP correspondente;
- Verificação de duplicidade de e-mail antes do registro, evitando contas duplicadas.

3.2.3 Fornecedor.php

Implementa o CRUD (Create, Read, Update, Delete) completo da entidade Fornecedor, sendo consumida pelos endpoints AJAX da pasta `/ajax`:

- cadastrar(dados): insere um novo fornecedor (razão social, CNPJ, contato, e-mail, etc.);
- listar() / buscarPorId(id): retorna todos os fornecedores ou um registro específico para exibição/edição;
- atualizar(id, dados): edita as informações de um fornecedor existente;
- excluir(id): remove um fornecedor, com tratamento de integridade referencial em relação aos produtos vinculados.

3.2.4 Produto.php

Implementa o CRUD completo da entidade Produto, mantendo o relacionamento obrigatório com um Fornecedor (chave estrangeira `fornecedor_id`):

- cadastrar(dados): insere um novo produto, associando-o a um fornecedor previamente cadastrado através de um campo de seleção (select);
- listar() / listarComFornecedor(): retorna os produtos, realizando um JOIN com a tabela `fornecedores` para exibir o nome do fornecedor junto ao produto;
- atualizar(id, dados) / excluir(id): operações padrão de edição e remoção;
- Validações de preço (numérico, maior que zero) e de vínculo obrigatório com um fornecedor válido.

3.2.5 Cesta.php

Concentra a lógica de negócio do carrinho de compras (cesta), incluindo o gerenciamento híbrido entre sessão PHP e persistência em banco de dados:

- adicionarItem(produto_id) / removerItem(produto_id): controla a seleção/deseleção de produtos via checkbox na interface;

- `listarItens()`: retorna os produtos atualmente selecionados na cesta do usuário logado;
- `calcularTotal()`: percorre os itens da cesta e soma os preços dos produtos selecionados, retornando o valor total formatado em Reais (R\$);
- `sincronizarComSessao()`: mantém a consistência entre o estado em `$_SESSION` e os registros da tabela `cesta_itens`, permitindo que a cesta seja recuperada mesmo após atualização da página.

4. DADOS E MODELAGEM (DATABASE)

4.1 Esquema Relacional

O banco de dados `mini_gestao_produtos` é composto por quatro tabelas principais, relacionadas entre si por meio de chaves estrangeiras, conforme representado esquematicamente a seguir:

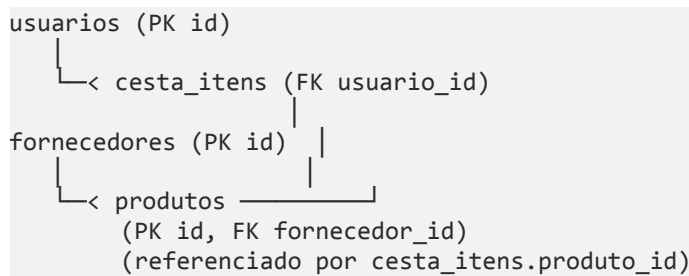


Figura 4.1 — Diagrama esquemático de relacionamento entre as tabelas

4.2 Descrição das Tabelas

4.2.1 Tabela usuarios

Campo	Tipo	Restrições
id	INT	PK, AUTO_INCREMENT
nome	VARCHAR(150)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
senha_hash	VARCHAR(255)	NOT NULL (SHA-256 + salt)
criado_em	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabela 4.1 — Estrutura da tabela usuarios

4.2.2 Tabela fornecedores

Campo	Tipo	Restrições
id	INT	PK, AUTO_INCREMENT
nome	VARCHAR(150)	NOT NULL
cnpj	VARCHAR(20)	UNIQUE
telefone	VARCHAR(20)	NULL
email	VARCHAR(150)	NULL
criado_em	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabela 4.2 — Estrutura da tabela fornecedores

4.2.3 Tabela produtos

Campo	Tipo	Restrições
id	INT	PK, AUTO_INCREMENT
nome	VARCHAR(150)	NOT NULL
descricao	TEXT	NULL
preco	DECIMAL(10,2)	NOT NULL
fornecedor_id	INT	FK → fornecedores(id), NOT NULL
criado_em	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabela 4.3 — Estrutura da tabela produtos

4.2.4 Tabela cesta_itens

Campo	Tipo	Restrições
id	INT	PK, AUTO_INCREMENT
usuario_id	INT	FK → usuarios(id), NOT NULL
produto_id	INT	FK → produtos(id), NOT NULL
quantidade	INT	DEFAULT 1
adicionado_em	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Tabela 4.4 — Estrutura da tabela cesta_itens

As chaves estrangeiras fornecedor_id (em produtos) e usuario_id / produto_id (em cesta_itens) garantem a integridade referencial do sistema, impedindo, por exemplo, o cadastro de um produto vinculado a um fornecedor inexistente ou a inserção de um item de cesta associado a um produto excluído.

5. FLUXO DE FUNCIONAMENTO DA APLICAÇÃO

A seguir, descreve-se o fluxo de utilização do sistema do ponto de vista do usuário final, do primeiro acesso até a conclusão da seleção de produtos na cesta de compras.

Passo 1 — Cadastro e Login no Sistema

O usuário acessa a tela de registro (auth/registrar.php) e informa nome, e-mail e senha. O back-end, através da classe Usuario.php, valida os dados, aplica o hash SHA-256 com salt sobre a senha e persiste o novo registro na tabela usuarios. Em seguida, o usuário realiza login (auth/login.php); a senha informada é hasheada novamente e comparada com o valor armazenado. Em caso de sucesso, uma sessão PHP é iniciada e o usuário é redirecionado ao dashboard (index.php). Todas as páginas subsequentes passam a exigir a validação de auth_check.php.

Passo 2 — Cadastro e Gestão de Fornecedores via AJAX

Na tela fornecedores.php, o usuário preenche um formulário (nome, CNPJ, telefone, e-mail) e o submete. O JavaScript (fornecedores.js) intercepta o evento de envio, impede o recarregamento padrão da página e envia os dados via Fetch API (método POST) para o endpoint ajax/fornecedor_salvar.php, que invoca a classe Fornecedor.php para persistir a informação. A resposta em JSON é utilizada para atualizar dinamicamente a listagem em tela (tabela Bootstrap), exibindo mensagens de sucesso ou erro sem recarregar a página. As operações de edição e exclusão seguem a mesma lógica assíncrona.

Passo 3 — Cadastro e Gestão de Produtos Vinculados aos Fornecedores

Na tela produtos.php, o formulário de cadastro inclui um campo select preenchido dinamicamente com a lista de fornecedores já cadastrados. Ao submeter o formulário, os dados (nome, descrição, preço e fornecedor_id) são enviados via AJAX ao endpoint ajax/produto_salvar.php, que aciona a classe Produto.php. A listagem de produtos exibe, para cada item, o nome do fornecedor correspondente, obtido por meio de uma consulta com JOIN entre as tabelas produtos e fornecedores.

Passo 4 — Seleção de Produtos por Checkbox

Cada produto listado é acompanhado de um checkbox. Ao marcar ou desmarcar um item, um evento JavaScript dispara uma requisição assíncrona ao endpoint ajax/cesta_atualizar.php, que utiliza a classe Cesta.php para adicionar ou remover o produto correspondente da cesta do usuário logado (persistida na tabela cesta_itens e refletida na sessão). Essa abordagem evita a necessidade de um botão “adicionar ao carrinho” separado, simplificando a experiência de uso.

Passo 5 — Visualização do Resumo da Cesta de Compras

Na tela cesta.php, o sistema recupera, através de Cesta.php, todos os itens atualmente selecionados pelo usuário, exibindo-os em uma tabela com nome do produto, fornecedor e preço unitário. O método calcularTotal() soma os valores dos itens selecionados e apresenta o resultado formatado em Reais (R\$), atualizado automaticamente sempre que a seleção de produtos é alterada em qualquer uma das telas do sistema.

5.1 Diagrama Resumido do Fluxo

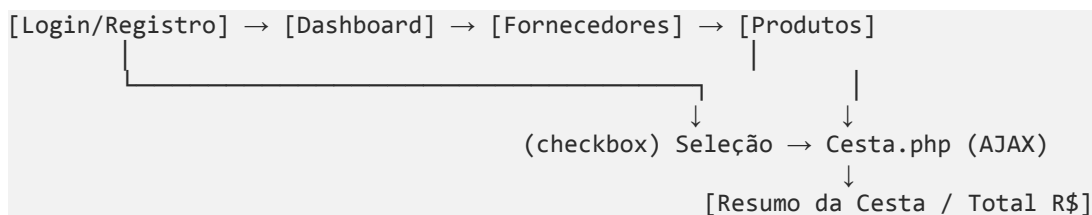


Figura 5.1 — Fluxo geral de navegação e interação do usuário

6. CONCLUSÃO

O sistema “Mini Gestão de Produtos” cumpre integralmente os requisitos funcionais estabelecidos para o projeto, oferecendo um fluxo coeso de gestão de fornecedores, produtos e composição de uma cesta de compras, com cálculo automático de valores em tempo real. A adoção de Programação Orientada a Objetos no back-end, aliada ao padrão de projeto Singleton na camada de conexão com o banco de dados, resultou em um código organizado, de fácil manutenção e com baixo acoplamento entre as responsabilidades.

A separação de responsabilidades entre front-end e back-end mostrou-se um dos pontos mais relevantes da arquitetura: enquanto o HTML/CSS/Bootstrap cuida exclusivamente da apresentação visual e da responsividade, e o JavaScript (Fetch API) é responsável pela orquestração das interações assíncronas, toda a regra de negócio permanece isolada nas classes PHP (Database, Usuario, Fornecedor, Produto e Cesta), o que facilita tanto a manutenção evolutiva quanto eventuais testes unitários futuros.

Do ponto de vista de segurança, o uso de Prepared Statements via PDO elimina a superfície de ataque mais comum em aplicações PHP legadas (SQL Injection), enquanto o hashing de senhas com SHA-256 e salt,

combinado à verificação de sessão centralizada em `auth_check.php`, garante um nível de proteção adequado ao escopo educacional e de implantação local do projeto.

Por fim, a criação automática do banco de dados e das tabelas no primeiro acesso torna o sistema imediatamente executável em qualquer ambiente local compatível com PHP e MySQL/MariaDB (como XAMPP, WAMP ou Laragon), sem exigir configuração manual de schema. Conclui-se que o sistema está pronto para implantação em ambiente local de testes e avaliação acadêmica, cumprindo seu propósito didático de demonstrar, de forma prática e integrada, conceitos de POO, modelagem relacional de dados, segurança da informação e comunicação assíncrona cliente-servidor.

Documento gerado como material de apoio técnico-acadêmico do projeto Mini Gestão de Produtos.